

PPS - Scala Stream Fusion

Leonardo Naddei

20 settembre 2026

Indice

1	Metaprogrammazione e generazione statica del codice in Scala 3	4
1.1	Multi-stage programming in Scala 3	4
1.2	Metaprogrammazione generativa e analitica	5
1.3	Dalle espressioni all'AST tipizzato	5
1.4	Tipi staged e tipi riflessivi	6
1.5	Beta reduction delle funzioni	6
1.6	Simboli, variabili generate e hygiene	7
2	Analisi dei requisiti	8
2.1	Requisiti di business	8
2.1.1	Colmare il divario tra espressività e prestazioni	8
2.1.2	Adozione a basso attrito	8
2.2	Requisiti funzionali utente	8
2.2.1	RFU-1 (Creazione dello Stream)	8
2.2.2	RFU-2 (Composizione funzionale)	8
2.2.3	RFU-3 (Esecuzione parallela esplicita)	9
2.2.4	RFU-4 (Raccolta dei risultati)	9
2.2.5	RFU-5 (Configurazione)	9
2.3	Requisiti funzionali di sistema	9
2.3.1	Requisiti negativi ed esclusioni (Out of Scope)	10
2.4	Requisiti non funzionali	10
3	Analisi tecnica	12
4	Design	13
4.1	Architettura	13
4.2	Design dell'API pubblica	14
4.2.1	Operazioni terminali	14
4.2.2	Creazione dello stream: <code>from</code>	16
4.2.3	Tipologie di stream	16
4.2.4	Operazione terminale <code>collect</code>	17
4.2.5	Configurazione compile-time e runtime	18
4.2.6	Import semplificato dell'API	19
4.3	Design della Stream IR	19
4.3.1	Struttura dell'albero	20
4.3.2	Rappresentazione di <code>flatMap</code>	20
4.4	Rappresentazione per la Code Generation	22
4.4.1	Valori	22
4.4.2	Operazioni	23

5	Sviluppo	25
5.1	Parsing	25
5.1.1	Parsing della stream	25
5.1.2	Collector	29
5.1.3	Parsing della configurazione statica	30
5.2	Enrichment	31
5.3	Traduzione in OPIr	31
5.4	Lowering	31
5.5	Testing	31
5.6	Benchmark	31
6	Retrospettiva	32

Abstract

Il presente lavoro descrive la realizzazione di una libreria per Scala 3 che consenta di definire pipeline di elaborazione secondo un paradigma dichiarativo, ottenendo al contempo prestazioni comparabili a quelle di codice imperativo specializzato manualmente.

Basandosi sul concetto di stream fusion, introdotto in Stream Fusion: From Lists to Streams to Nothing at All, e sfruttando le funzionalità di metaprogrammazione offerte dalle macro di Scala 3, è stata sviluppata una libreria capace di trasformare pipeline dichiarative in codice specializzato ed efficiente generato a compile-time. La libreria offre inoltre un'architettura estendibile per la definizione di nuovi operatori terminali, permettendo di integrare nuovi comportamenti nella fase di fusione senza rinunciare alle ottimizzazioni statiche.

Tale generazione statica permette di eliminare i principali overhead tipici delle astrazioni funzionali su JVM. L'analisi del bytecode prodotto conferma la completa fusione delle operazioni sequenziali in un unico ciclo imperativo e l'inlining diretto delle lambda nel callsite. Ciò azzerava l'uso di istruzioni `invokedynamic`, elimina il dispatch indiretto (evitando call site polimorfici o megamorfici) e rimuove le allocazioni di collezioni intermedie, iteratori e closure, azzerando al contempo il boxing dei tipi primitivi.

Per certi aspetti, il modello ottenuto richiama il paradigma di genericità dei template C++: il codice viene specializzato in fase di compilazione rispetto ai tipi e alla specifica composizione delle operazioni, quando tali informazioni sono disponibili staticamente. Le macro di Scala aggiungono tuttavia una componente esplicita di metaprogrammazione, permettendo non soltanto di istanziare codice generico, ma anche di analizzare, trasformare e ottimizzare la struttura della pipeline prima della sua emissione.

I benchmark condotti tramite JMH mostrano uno speedup significativo e un consumo di memoria drasticamente ridotto rispetto alle librerie standard. Il risultato è un framework che preserva l'espressività, la composizionalità e la type safety tipiche della programmazione funzionale.

Capitolo 1

Metaprogrammazione e generazione statica del codice in Scala 3

Il sistema di metaprogrammazione di Scala 3 segue un approccio di *multi-stage programming*: una parte del programma viene eseguita durante uno stage precedente e produce codice destinato a uno stage successivo. Questo modello permette di costruire macro che generano codice ben tipato e safe per costruzione [?, ?].

Nel progetto sviluppato, la metaprogrammazione viene utilizzata in entrambe le sue forme principali: analitica, per ispezionare e destrutturare la pipeline scritta dall'utente, e generativa, per produrre il codice.

1.1 Multi-stage programming in Scala 3

Le due primitive fondamentali del sistema di metaprogrammazione staged di Scala 3 sono le *quote* e gli *splice*.

Una quote

```
1 '{ expression }
```

costruisce una rappresentazione tipizzata del codice contenuto al suo interno. Se **expression** produce un valore di tipo **T**, il risultato della quote ha tipo

```
1 Expr[T]
```

e rappresenta codice che produrrà un valore di tipo **T** nello stage successivo.

Uno splice:

```
1 ${ expression }
```

opera invece nella direzione opposta e inserisce all'interno del codice quotato il frammento di programma rappresentato da un'espressione **Expr[T]**.

Il classico esempio di generazione staged è l'elevamento a potenza con esponente noto durante la compilazione:

```
1 def powCode(x: Expr[Int], n: Int)(using Quotes): Expr[Int] =  
2   if n == 0 then  
3     Expr(1)  
4   else  
5     '{ $x * ${ powCode(x, n - 1) } }
```

La ricorsione di `powCode` viene eseguita durante l'espansione della macro. Per un esponente pari a tre, il frammento generato è concettualmente equivalente a

```
1 x * x * x * 1
```

e non contiene quindi né il ciclo né la ricorsione impiegata dal metaprogramma per costruirlo.

1.2 Metaprogrammazione generativa e analitica

Una macro puramente generativa costruisce un nuovo frammento di programma a partire da informazioni note durante la compilazione. Nel progetto è tuttavia necessario anche compiere l'operazione inversa: ricevere un frammento di programma, riconoscerne la struttura e decomporlo nei suoi componenti.

Scala 3 supporta questa forma di metaprogrammazione attraverso il pattern matching sul codice quotato. Ad esempio, una generica espressione può essere analizzata con un pattern della forma

```
1 expr match {  
2   case '{ someFunction($argument) } =>  
3     ...  
4 }
```

Per i casi più semplici sarebbe possibile eseguire questa analisi utilizzando esclusivamente quoted pattern. Nel progetto, tuttavia, è necessario accedere a dettagli più fini dell'albero generato dal compilatore, rendendo necessario il passaggio alla Reflection API.

1.3 Dalle espressioni all'AST tipizzato

Il tipo `Expr[T]` offre un'interfaccia ad alto livello sul codice. Tale rappresentazione nasconde parte della struttura interna dell'espressione. Quando è necessario analizzare direttamente il programma è possibile convertire una `Expr[T]` nel relativo nodo dell'AST prodotto dal compilatore:

```
1 expr.asTerm
```

La Reflection API espone quindi la struttura tipizzata prodotta dal compilatore. Tra i nodi più importanti utilizzati nel parser vi sono:

- `Apply`, per l'applicazione di una funzione;
- `TypeApply`, per un'applicazione contenente parametri di tipo;
- `Select`, per la selezione di un membro;
- `Ident`, per il riferimento a un identificatore;
- `Block`, per un blocco contenente dichiarazioni e un'espressione finale;
- `Inlined`, per rappresentare il risultato di un'espansione inline;
- `ValDef`, `Assign` e `Ref`, utilizzati durante la generazione di nuovo codice.

Questo livello di rappresentazione è sostanzialmente più espressivo rispetto alle sole quote.

1.4 Tipi staged e tipi riflessivi

Scala 3 mette a disposizione due rappresentazioni complementari:

Type[T] rappresenta il tipo *T* nell'API staged ed è utilizzabile insieme a **Expr[T]**, quote e splice;

TypeRepr rappresenta invece un tipo all'interno della Reflection API e permette di ispezionarne direttamente la struttura.

Data una **TypeRepr**, il metodo **.asType** restituisce un valore **Type[?]** sul quale è possibile eseguire pattern matching:

```
1 item.asType match {  
2   case '[elem] => ...  
3 }
```

Un esempio è l'estrazione del tipo per la creazione di un iteratore da un singolo elemento:

```
1 private def createSourceOf(item: Term): ParsedTree = {  
2   item.asType match {  
3     case '[elem] =>  
4       val singleExpr = item.asExprOf[elem]  
5       val iterableExpr: Expr[Iterable[elem]] =  
6         '{ Iterable.single[elem]({ singleExpr }) }  
7       ...  
8   }  
9 }
```

Questo esempio contiene in pochi passaggi l'intero collegamento tra reflection e staged programming:

1. viene determinato il tipo del termine dell'AST ricevuto;
2. il tipo viene estratto in **elem** mediante il pattern **'[elem]**;
3. il **Term** originale viene convertito in **Expr[elem]**;
4. viene generato un nuovo frammento **Expr[Iterable[elem]]**;

1.5 Beta reduction delle funzioni

Un altro elemento importante è l'utilizzo della β -reduction, considerando una funzione:

$$f = x \mapsto e, \quad (1.1)$$

l'applicazione

$$f(a) \quad (1.2)$$

può essere riscritta sostituendo direttamente *x* con *a* all'interno di *e*.

Scala 3 rende disponibile questa trasformazione attraverso: **Expr.betaReduce(...)**, viene utilizzata per l'applicazione esplicita delle lambda eliminando invocazioni tramite **invokedynamic**. Prendendo ad esempio l'operazione **filter**:

```
1 val cond = Expr.betaReduce('{ ${ filter.predicate }($elem) })
```

Se la funzione ricevuta è una lambda direttamente disponibile alla macro, il suo corpo può quindi essere incorporato nel codice generato.

Una pipeline come

```
1 stream
2   .filter(x => x > 0)
3   .map(x => x * 2)
```

può pertanto contribuire a generare un corpo concettualmente equivalente a

```
1 if (elem > 0) {
2   emit(elem * 2)
3 }
```

Invece che:

```
1 if (filter.test(elem)) {
2   emit(map.apply(elem))
3 }
```

Nell'ultimo caso, `filter` e `map` denotano istanze materializzate dalla JVM tramite call site `invokedynamic`, la loro applicazione avviene attraverso una chiamata indiretta all'interfaccia funzionale, ad esempio `Function1.apply`. Questo introduce un call site il cui target deve essere risolto e, qualora diventi polimorfo o megamorfo, può ridurre l'efficacia di devirtualizzazione, inlining e ulteriori ottimizzazioni interprocedurali del JIT. Incorporando staticamente il corpo della lambda nell'IR generato, la macro elimina sia la rappresentazione della funzione come entità invocabile sia il relativo dispatch.

1.6 Simboli, variabili generate e hygiene

Nell'AST di Scala, un riferimento a una variabile è associato semanticamente al relativo `Symbol`. Il nome è solamente una delle proprietà del simbolo e non ne rappresenta l'identità. La Reflection API permette di creare nuove variabili attraverso: `Symbol.newVal(...)`, e di fare successivamente riferimento allo stesso simbolo attraverso `Ref(symbol)`. La definizione viene creata nell'AST tramite `ValDef(symbol, Some(initializer))`

La distinzione tra nome e simbolo garantisce inoltre l'*hygiene* della macro: variabili generate dalla macro non possono andare in conflitto con variabili omonime definite nel codice utente.

TODO: esempio creazione e uso

Capitolo 2

Analisi dei requisiti

2.1 Requisiti di business

2.1.1 Colmare il divario tra espressività e prestazioni

La programmazione funzionale favorisce la leggibilità, l'astrazione e la manutenibilità del codice attraverso l'impiego di pipeline dichiarative. Tuttavia, sulla JVM tali astrazioni comportano comunemente una penalizzazione prestazionale rispetto a cicli imperativi scritti a mano. L'obiettivo primario del progetto è annullare questo compromesso, consentendo di scrivere codice ad alto livello dichiarativo che compili direttamente in codice a basso livello massimamente efficiente.

2.1.2 Adozione a basso attrito

L'integrazione della libreria all'interno di progetti esistenti non deve richiedere modifiche al toolchain standard del compilatore né l'apprendimento di API complesse o esotiche. La sintassi offerta deve risultare familiare a chi già utilizza le collezioni standard di Scala o le API di Java Stream, garantendo un'interfaccia compatta, uniforme e accessibile tramite un singolo import.

2.2 Requisiti funzionali utente

2.2.1 RFU-1 (Creazione dello Stream)

L'utente deve poter istanziare un flusso di elaborazione a partire da diverse sorgenti di dati (ad esempio collezioni Scala, array primitivi o iteratori) attraverso un unico punto di ingresso intuitivo (`FusedStream.from(...)`).

2.2.2 RFU-2 (Composizione funzionale)

L'utente deve poter comporre la sequenza di trasformazione mediante un'interfaccia fluida e concatenabile, disponendo delle principali operazioni di flusso:

- Trasformazioni elemento per elemento (`map`);
- Filtri (`filter`);
- Operazioni di campionamento e troncamento (`skip`, `limit`);
- Appiattimento e combinazione monadica di flussi interni (`flatMap`).

2.2.3 RFU-3 (Esecuzione parallela esplicita)

Qualora la sorgente dei dati sia adatta, l'utente deve poter richiedere l'esecuzione concorrente della pipeline invocando esplicitamente il metodo `.parallel()`.

2.2.4 RFU-4 (Raccolta dei risultati)

L'utente deve poter raccogliere il risultato di una stream tramite `.collect(...)`, potendo utilizzare:

- Collector predefiniti della libreria (come `toList`, `toArray`, `summing`, `findFirst`);
- Collector personalizzati che rispettino l'interfaccia di accumulazione estendibile fornita dal framework.

2.2.5 RFU-5 (Configurazione)

L'utente deve poter sovrascrivere il comportamento predefinito del sistema, sia per quanto riguarda i parametri di generazione statica del codice che per i parametri riguardanti l'esecuzione a runtime. Questo deve essere opzionale e deve avere attrito minimo per l'utilizzo

RFU-6 (Guida contestuale tramite Type System)

L'API deve guidare attivamente l'utente durante la costruzione della pipeline sfruttando il type system di Scala. Tramite tipi opachi, raffinamenti e sottotipazione avanzata, l'IDE e il compilatore devono esporre (tramite auto-completamento) ed accettare solo ed esclusivamente le operazioni valide per lo stato corrente dello stream (es. inibendo metodi non supportati in parallelo o impedendo l'uso di collector incompatibili con la policy di terminazione).

2.3 Requisiti funzionali di sistema

RFS-1 (Riconoscimento e analisi della pipeline statica): Il sistema deve essere in grado di ispezionare l'albero sintattico di una pipeline dichiarativa e scomporla nella sequenza logica di sorgente, trasformazioni intermedie e operazione terminale a compile time.

RFS-2 (Fusione delle operazioni sequenziali): Il sistema deve collassare le trasformazioni concatenate in un unico flusso di esecuzione continuo, eliminando la materializzazione di collezioni o iteratori tra uno stadio e il successivo.

RFS-3 (Generazione di codice specializzato): Il sistema deve emettere codice imperativo target direttamente specializzato per i tipi di dato coinvolti e per l'operazione terminale richiesta, evitando chiamate polimorfiche o indirette alle funzioni utente.

RFS-4 (Gestione del flusso e terminazione anticipata): Il sistema deve regolare il flusso di controllo del ciclo generato per supportare sia l'elaborazione esaustiva dell'intero flusso sia l'interruzione anticipata (*short-circuiting*) quando la logica dell'operazione terminale o di filtro lo richiede.

RFS-5 (Composizione di flussi annidati): Il sistema deve consentire la fusione di flussi generati dinamicamente da elementi intermedi (operazioni monadiche/`flatMap`), integrando il ciclo interno nel contesto del ciclo esterno senza creare buffer di accumulazione intermedi.

RFS-6 (Supporto all'esecuzione parallela): Per le sorgenti e i terminali compatibili, il sistema deve emettere codice in grado di suddividere il carico di lavoro su più worker e aggregarne i risultati intermedi.

2.3.1 Requisiti negativi ed esclusioni (Out of Scope)

Al fine di garantire la terminazione del metaprogramma, la predicibilità prestazionale e la robustezza del compilatore, sono definiti i seguenti requisiti negativi:

- **RN-1 (Assenza di composizione dinamica runtime):** Il sistema non deve supportare la costruzione dinamica di pipeline determinate da rami condizionali ordinari a runtime (ad esempio `if (cond) s.map(...) else s.filter(...)`). L'intera catena di trasformazioni deve essere fissa e nota sintatticamente al call-site di compilazione.
- **RN-2 (Nessuna ottimizzazione opaca sul codice utente):** Il sistema non deve alterare né riscrivere la logica interna delle espressioni arbitrarie fornite dall'utente all'interno delle lambda, limitandosi all'inlining del corpo nel call-site generato.
- **RN-3 (Nessun supporto a backtracking o accesso casuale):** Gli stream sono modellati come flussi unidirezionali in avanti (*forward-only* / push-based). Non è richiesto il supporto ad accessi casuali all'indietro o alla rilettura dello storico di elementi già consumati.
- **RN-4 (Nessuna distribuzione su cluster):** L'ambito di concorrenza è limitato alla memoria condivisa mononodo (JVM locale); non è contemplata la serializzazione o la distribuzione di calcoli su nodi remoti.
- **RN-5 (Nessun fall-back silente a runtime):** Se una composizione di stream viola i requisiti statici o non è compatibile con la fusione, il compilatore deve rigettare esplicitamente il programma con un errore a compile-time, senza ricorrere a conversioni lente o fallback a runtime non dichiarati.
- **RN-6 (Nessun inlining o ottimizzazione per funzioni non-lambda):** Qualora a una trasformazione (es. `map`) venga passato un riferimento opaco a una funzione preesistente o un method value anziché un'espressione lambda letterale, il compilatore non applica la β -reduction né l'inlining statico; la chiamata viene degradata a normale invocazione indiretta dell'interfaccia funzionale.
- **RN-7 (Nessun supporto a flussi nidificati di stream):** Il sistema non supporta la generazione di stream i cui singoli elementi siano a loro volta istanze di stream tramite trasformazioni proiettive ordinarie (es. `map(x => FusedStream.of(x))`). La sola composizione nidificata consentita è quella finalizzata all'appiattimento esplicito tramite `flatMap`.
- **RN-8 (Nessuna distribuzione su cluster):** L'ambito di concorrenza è strettamente limitato alla memoria condivisa mononodo (JVM locale); non è contemplata la serializzazione o la distribuzione di calcoli su cluster remoti.
- **RN-9 (Nessun fall-back silente a runtime):** Se una composizione di stream viola i vincoli di tipo o non è compatibile con la fusione, il compilatore deve rigettare esplicitamente il programma tramite un errore di tipo statico o errore di compilazione della macro, senza ricorrere a conversioni lente o fallback trasparenti a runtime.

2.4 Requisiti non funzionali

RNF-1 (Prestazioni a runtime)

Il codice imperativo generato deve esibire un throughput computazionale comparabile a cicli imperativi scritti manualmente in Scala o Java, riducendo l'overhead computazionale rispetto alle pipeline funzionali convenzionali ed eliminando il costo del boxing per i tipi primitivi.

RNF-2 (Allocazione di memoria zero-overhead)

L'elaborazione delle operazioni intermedie stateless non deve allocare oggetti effimeri sul runtime heap (nessun iteratore, nessuna closure associata a lambda, nessun contenitore intermedio).

RNF-3 (Type Safety statica e Guida del Compilatore)

Tutte le combinazioni illegali di operazioni — come l'impiego di collector *short-circuiting* non combinabili in una pipeline parallela o l'invocazione di `skip` e `limit` su stream paralleli — devono essere impediti e segnalati dal type system di Scala a compile-time, garantendo che lo sviluppatore sia guidato dai tipi verso pipeline formalmente corrette prima ancora dell'esecuzione.

RNF-4 (Igiene delle Macro)

Tutte le entità introdotte dal generatore di codice (variabili ausiliarie, indici di ciclo, flag di terminazione) devono essere modellate tramite simboli univoci e isolati (*hygienic macro expansion*), garantendo l'assoluta assenza di conflitti semantici o shadowing con identificatori presenti nel codice utente.

Capitolo 3

Analisi tecnica

Sintesi delle Operazioni e Proprietà Algebriche

In Tabella 3.1 viene riportato un quadro riassuntivo delle firme formali e delle caratteristiche strutturali delle operazioni analizzate.

Tabella 3.1: Confronto formale delle operazioni trasformative su flussi di dati.

Operazione	Firma di Dominio	Tipo di Trasformazione	Cardinalità ($1 : M$)	Gestione Stato
Map	$A \rightarrow B$	$\mathcal{S}(A) \rightarrow \mathcal{S}(B)$	$1 : 1$	Stateless
Filter	$A \rightarrow \mathbb{B}$	$\mathcal{S}(A) \rightarrow \mathcal{S}(A)$	$1 : \{0, 1\}$	Stateless
Limit	$\mathbb{N}_0$	$\mathcal{S}(A) \rightarrow \mathcal{S}(A)$	$1 : \{0, 1\}$	Stateful
Skip	$\mathbb{N}_0$	$\mathcal{S}(A) \rightarrow \mathcal{S}(A)$	$1 : \{0, 1\}$	Stateful
FlatMap	$A \rightarrow \mathcal{S}(B)$	$\mathcal{S}(A) \rightarrow \mathcal{S}(B)$	$1 : N \ (N \geq 0)$	Stateless / Contextual

algoritmo implementato (push stream)

ottimizzazioni applicate ai parametri

enrichement: bubbling delle dichiarazioni

generazione di codice

implementazione del filter

implementazione del flatmapping

es semplice:

IncIter(from: Int): infinite lazy iterator, each next returns the next

FusedStream.from(List(1,2,3,4,5)).skip(3).flatMap(n -> Stream.of(IncIter(n*10)).limit(3))

.limit(10).toList()

// Must be converted to

i = 0 j = 0

for i in l: if(i++ >= 3) «< must inject condition on j too k = 0 iter = IncIter(i* 10) for

ii in iter if(k++ >= 3) break ——— This get injectd into the flatMap ——— if(j ++ >= 10)

break all push(ii) ———

esempio estensione con collector utilizzando i trait

Capitolo 4

Design

4.1 Architettura

L'architettura del compilatore è organizzata come una sequenza di trasformazioni che partono da un AST Scala e terminano con un, nuovo, AST Scala. Come mostrato in Figura 4.1, è possibile distinguere concettualmente il processo in un frontend e in un backend.

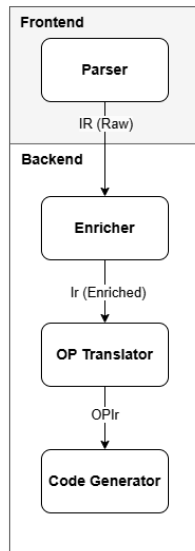


Figura 4.1: Architettura generale del compilatore di FusedStream.

L'applicativo è composto da due aree: l'API utilizzabile (vedi 4.2) e un compilatore che si occupa del parsing a creazione di codice ad hoc a partire dall'albero sintattico di una macro.

Il *frontend* ha il compito di analizzare l'espressione Scala della pipeline e di ricostruirne la struttura in una rappresentazione logica interna. Il parser riconosce le sorgenti e gli operatori messi a disposizione dalla DSL e produce una prima rappresentazione della pipeline, denominata *Stream IR*. In questa fase la rappresentazione mantiene ancora una corrispondenza diretta con i concetti di alto livello del modello di streaming, quali sorgenti, operazioni.

Il *backend* riceve la rappresentazione prodotta dal frontend e la trasforma progressivamente fino a ottenere il codice che implementa concretamente la pipeline. La prima fase consiste nell'*enrichment* della *Stream IR*. Durante questa trasformazione vengono associate all'albero informazioni necessarie alle fasi successive, tra cui variabili ausiliarie, condizioni di terminazione e proprietà della pipeline. La struttura risultante è un *Enriched Stream IR*.

La *Enriched Stream IR* conserva ancora la semantica degli operatori di stream, ma contiene informazioni sufficienti per determinare una strategia di esecuzione. Essa viene quindi sottoposta

a una seconda trasformazione, che produce la *Operation IR* (OP IR). Quest'ultima è costituita da operazioni più vicine al codice imperativo da generare come dichiarazioni di variabili, assegnamenti, condizioni, cicli, accessi ad array e operazioni necessarie all'esecuzione parallela.

L'ultima fase è costituita dal *code generator*, che effettua il lowering della OP IR verso costrutti Scala concreti. A differenza di un compilatore tradizionale, il risultato è un nuovo AST Scala. Tale AST viene restituito al compilatore Scala e reinserito nel normale processo di compilazione del programma.

Il flusso complessivo della trasformazione può quindi essere sintetizzato come:

Scala AST $\longrightarrow$ Stream IR_{Raw} $\longrightarrow$ Stream IR_{Enriched} $\longrightarrow$ OP IR $\longrightarrow$ Generated Scala AST.

4.2 Design dell'API pubblica

Particolare attenzione è stata posta nella progettazione dell'API pubblica della libreria. L'obiettivo è stato quello di fornire un'interfaccia che garantisca contemporaneamente:

- la possibilità di descrivere pipeline sufficientemente espressive
- l'estendibilità, in particolare per quanto riguarda le operazioni terminali
- il maggior numero possibile di garanzie a compile time
- la minimizzazione del codice necessario per verificare dinamicamente la correttezza delle combinazioni di operazioni richieste dall'utente.

Dal punto di vista concettuale, ogni stream può essere scomposto in tre componenti principali:

1. una sorgente, generalmente costituita da una collezione o da un array;
2. una sequenza di operazioni intermedie (`map`, `filter`, `flatMap`,...)
3. un'operazione terminale

La gerarchia di tipi esposta dall'API riflette direttamente questa struttura. L'utilizzo di trait sealed, tipi parametrici, tipi opachi e vincoli sui parametri di tipo permette di rappresentare a livello statico le proprietà della pipeline. In questo modo il compilatore Scala può guidare l'utente durante la costruzione dello stream e impedire le combinazioni non supportate.

Allo stesso tempo, l'API mantiene un certo grado di estendibilità, in particolare consentendo all'utente di definire nuove operazioni terminali attraverso l'interfaccia dei collector.

4.2.1 Operazioni terminali

Le operazioni terminali sono rappresentate attraverso l'astrazione `Collector`. La sua definizione generale è:

```
1 trait Collector[E, B, R, S <: TerminationPolicy] extends CollectorBase[E, B, R]
```

I quattro parametri di tipo rappresentano rispettivamente:

- E: il tipo degli elementi ricevuti dallo stream;
- B: il tipo dello stato intermedio, o buffer, utilizzato durante l'accumulazione;

- R: il tipo del risultato finale prodotto dal collector;
- S: la politica di terminazione del collector.

Il comportamento fondamentale di un collector è definito da `CollectorBase`:

```
1 trait CollectorBase[E, B, R] {
2   def supplier(): B
3   def accumulator(buf: B, elem: E): Boolean
4   def finisher(buf: B): R
5 }
```

Il metodo `supplier` crea lo stato iniziale utilizzato durante la raccolta, mentre `accumulator` aggiunge un nuovo elemento, il valore booleano restituito segnala che l'elaborazione può essere interrotta (si pensi a una `findFirst`). Infine, `finisher` trasforma lo stato intermedio nel risultato finale.

La politica di terminazione è rappresentata mediante la seguente gerarchia:

```
1 sealed trait TerminationPolicy
2 sealed trait Exhaustive extends TerminationPolicy
3 sealed trait ShortCircuiting extends TerminationPolicy
```

Un collector `Exhaustive` richiede l'elaborazione dell'intero upstream, mentre un collector `ShortCircuiting` può terminare non appena viene soddisfatta una determinata condizione.

Ad esempio, i collector `toList` e `findFirst` hanno rispettivamente i seguenti tipi:

```
1 def toList[T]: Collector[T, ListBuffer[T], List[T], Exhaustive]
2 def findFirst[T]: Collector[T, OptionBuffer[T], Option[T], ShortCircuiting]
```

Nel primo caso è necessario consumare tutti gli elementi dello stream prima di produrre la lista risultante. Nel secondo caso, invece, il primo elemento ricevuto è sufficiente a determinare il risultato e il collector può richiedere l'interruzione anticipata della pipeline. Questa proprietà viene codificata direttamente nel tipo del collector.

Collector combinabili e parallelismo

Per poter eseguire una riduzione in parallelo non è sufficiente che il collector sia esaustivo. Gli stati intermedi prodotti indipendentemente dalle diverse partizioni dello stream devono anche poter essere combinati, questa proprietà viene rappresentata dall'interfaccia:

```
1 trait Combinable[B] {
2   def combine(left: B, right: B): B
3 }
```

Un collector utilizzabile come terminale di una pipeline parallela viene quindi definito come:

```
1 trait ParallelCollector[E, B, R]
2   extends Collector[E, B, R, Exhaustive]
3   with Combinable[B]
```

Il tipo `ParallelCollector` esprime pertanto staticamente due proprietà:

1. la necessità di elaborare completamente lo stream;
2. la possibilità di combinare i risultati intermedi ottenuti da più worker.

Il type system permette così di distinguere un collector genericamente valido in una pipeline sequenziale da un collector che possiede anche le proprietà necessarie per essere utilizzato in una pipeline parallela.

Collector specializzati

Alcune operazioni terminali particolarmente comuni dispongono di un trattamento specializzato da parte del compilatore di FusedStream. Due esempi sono `toArray` e `summing`:

```
1 opaque type ToArrayCollector[T] <: ParallelCollector
2 opaque type SummingCollector[T <: Summable] <: ParallelCollector
```

I relativi metodi sono inoltre annotati con `@compileTimeOnly`:

Questi collector non vengono utilizzati come normali implementazioni runtime di `Collector`. La loro presenza nella pipeline viene riconosciuta durante l'espansione della macro e utilizzata per selezionare una strategia di code generation dedicata.

Il tipo opaco permette di esporre verso l'esterno soltanto le proprietà rilevanti del collector. Ad esempio, entrambi risultano essere sottotipi di `ParallelCollector`, rendendoli compatibili con le pipeline parallele, senza esporre all'utente i dettagli della strategia utilizzata internamente dal compilatore.

La combinazione di tipi opachi e `@compileTimeOnly` permette quindi di utilizzare il tipo del collector come protocollo di comunicazione tra l'API pubblica e la macro di compilazione.

Nel caso di `toArray`, il compilatore può generare direttamente una strategia di accumulazione specializzata evitando, quando possibile, boxing e chiamate indirette attraverso l'interfaccia `Collector`. Analogamente, `summing` può essere trasformato in una riduzione diretta.

In questo modo l'API mantiene una sintassi uniforme:

```
1 stream.collect(Collector.toArray)
2 stream.collect(Collector.summing)
```

pur consentendo al compilatore di generare implementazioni sostanzialmente differenti e specializzate per le singole operazioni terminali.

4.2.2 Creazione dello stream: `from`

La costruzione di una pipeline avviene a partire dal metodo `from`. Oltre a identificare la sorgente degli elementi, il tipo restituito da `from` determina quali modalità di esecuzione possono essere richieste successivamente.

Non tutte le sorgenti possiedono infatti le caratteristiche necessarie per essere sfruttate parallelamente. L'API rappresenta questa proprietà attraverso il tipo restituito: una sorgente per la quale è disponibile l'esecuzione parallela viene rappresentata come un `ParallelizableSource[A]` (vedi 4.2).

Il metodo `parallel` non appartiene quindi genericamente a ogni `Stream`, ma solamente alle sorgenti per le quali la libreria dichiara esplicitamente il supporto alla parallelizzazione. Una pipeline costruita a partire da una sorgente non parallelizzabile non può essere convertita in una `ParallelStream`, l'operazione semplicemente non è disponibile nel tipo statico restituito all'utente.

4.2.3 Tipologie di stream

La modalità di esecuzione della pipeline è rappresentata attraverso una gerarchia di tipi `sealed`:

```
1 sealed trait Stream[A]
2 sealed trait SequentialStream[A] extends Stream[A]
3 sealed trait ParallelizableSource[A] extends SequentialStream[A]
4 sealed trait ParallelStream[A] extends Stream[A]
```

`Stream[A]` rappresenta l'insieme minimo delle operazioni disponibili indipendentemente dalla modalità di esecuzione:

```

1 def filter(p: A => Boolean): Stream[A]
2 def map[B](f: A => B): Stream[B]
3 def flatMap[B](f: A => SequentialStream[B]): Stream[B]

```

I sottotipi ridefiniscono (in maniera covariantemente) il tipo restituito dalle operazioni, ad esempio:

```

1 sealed trait ParallelStream[A] extends Stream[A] {
2   override def filter(p: A => Boolean): ParallelStream[A]
3   override def map[B](f: A => B): ParallelStream[B]
4   override def flatMap[B](f: A => SequentialStream[B]): ParallelStream[B]
5 }

```

Una chiamata a `map` su una `ParallelStream[A]` produce quindi una `ParallelStream[B]`, impedendo che l'informazione relativa alla modalità di esecuzione venga persa lungo la pipeline.

Analogamente, una pipeline sequenziale:

```

1 sealed trait SequentialStream[A] extends Stream[A] {
2   override def filter(p: A => Boolean): SequentialStream[A]
3   override def map[B](f: A => B): SequentialStream[B]
4   override def flatMap[B](f: A => SequentialStream[B]): SequentialStream[B]
5   // Non supportati in stream paralleli
6   def skip(n: Int): SequentialStream[A]
7   def limit(n: Int): SequentialStream[A]
8 }

```

La posizione di `skip` e `limit` esclusivamente all'interno di `SequentialStream` garantisce che queste operazioni non risultino disponibili su una `ParallelStream`.

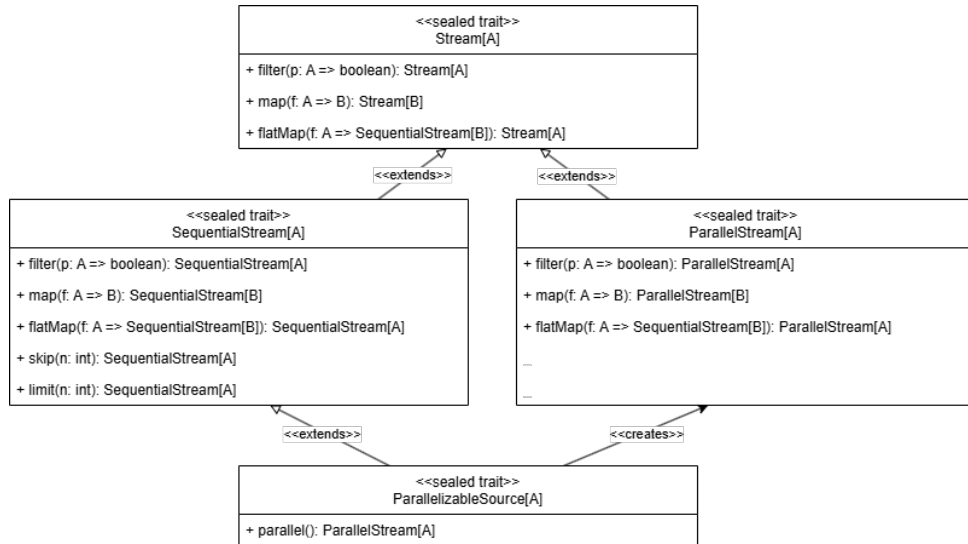


Figura 4.2: Diagramma UML delle classi per la gerarchia di Stream.

4.2.4 Operazione terminale collect

L'operazione `collect` costituisce il punto nel quale la descrizione della pipeline viene trasformata nel codice che ne realizza l'esecuzione.

La distinzione statica tra `SequentialStream` e `ParallelStream` permette di imporre requisiti differenti sul collector terminale.

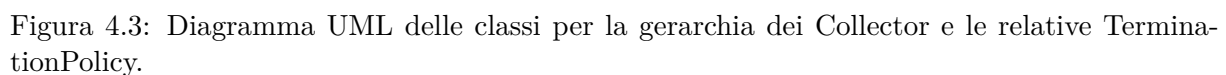
```
1 // Sequential stream
2 collect(collector: Collector[E, B, R, S]): R
3
4 // Parallel stream
5 collect(collector: ParallelCollector[E, B, R]): R
```

Ad esempio, un'operazione come:

non soddisfa il vincolo richiesto da `ParallelCollector`, poiché `findFirst` è tipato come:

Al contrario:

è accettabile perché `ToArrayCollector[T]` è dichiarato come sottotipo di `ParallelCollector`.



La libreria distingue tra due tipologie di configurazione: `CompileConfig`, relativa alle scelte che influenzano la generazione del codice, e `RuntimeConfig`, relativa ai parametri utilizzati durante l'esecuzione.

Entrambe le configurazioni sono gestite tramite il meccanismo dei **given** di Scala. La libreria fornisce un comportamento di default, permettendo quindi di utilizzare `FusedStream` senza alcuna configurazione esplicita nei casi più comuni.

Quando necessario, l'utente può comunque sovrascrivere i valori predefiniti fornendo una propria istanza **given** della configurazione desiderata nel contesto della pipeline.

In questo modo l'API mantiene una configurazione implicita e poco invasiva, lasciando comunque la possibilità di personalizzare sia il comportamento in fase di compilazione sia quello in fase di esecuzione.

4.2.6 Import semplificato dell'API

Gli elementi dell'API pubblica vengono riesposti direttamente attraverso l'oggetto `FusedStream`. Questo permette di importare l'intera API necessaria con una singola istruzione:

```
1 import FusedStream.*
```

ottenendo così accesso sia alle funzioni per la costruzione delle pipeline sia ai collector terminali pronti all'uso. L'obiettivo è mantenere l'interfaccia compatta e immediata, evitando che l'utente debba conoscere o importare esplicitamente i diversi oggetti interni che compongono l'API.

4.3 Design della Stream IR

L'obiettivo della *Stream Intermediate Representation* (Stream IR) è rappresentare la struttura logica di una pipeline e, permettere alle successive fasi del compilatore di associare all'albero nuove informazioni.

Il design della Stream IR è una struttura ad albero estensibile che prende ispirazione dal paper *Trees That Grow* [1].

Il problema principale da gestire è che un Abstract Syntax Tree non rimane immutato durante l'intero processo di compilazione. Al contrario, le fasi del compilatore introducono informazioni da associare ai nodi dell'albero. Una possibile soluzione consiste nel definire una rappresentazione differente per ciascuna fase. Questo porterebbe però duplicazione di una parte considerevole della struttura dell'AST. Un'altra opzione potrebbe essere avere un nodo unico per ogni fase con campi opzionali o un nodo base con varie estensioni per reappresentare le proprie fasi, questi approcci hanno però un problema: rendono più complicata e soggetta a possibili errori la gestione di questi dati.

E' stata invece creata una struttura parametrizzata a livello di tipo P per determinare fase alla quale appartiene l'albero.

```
1 sealed trait Phase
2 object Phase {
3   sealed trait Raw extends Phase
4   sealed trait Enriched extends Phase
5 }
6 enum StreamTree[P <: Phase, A] {
7   ...
8 }
```

A Rappresenta il tipo degli elementi prodotti dal nodo.

P Identifica la fase del ciclo di vita (**Phase**) nella quale il nodo si trova.

L'utilizzo della fase come parametro di tipo, rende esplicito al type system quali rappresentazioni possono essere manipolate, ne derivano benefici come pattern matching esaustivo e garanzie a compile time.

Il passaggio tra il risultato del parser e dell'enricher quindi viene espresso direttamente attraverso i tipi.

`StreamTree[Phase.Raw, A] StreamTree[Phase.Enriched, A].`

4.3.1 Struttura dell'albero

La Stream IR modella una pipeline come un albero i cui nodi corrispondono alle operazioni del dominio degli stream. Le sorgenti costituiscono la foglia dell'albero.

Una rappresentazione semplificata della gerarchia è mostrata in Figura 4.4.

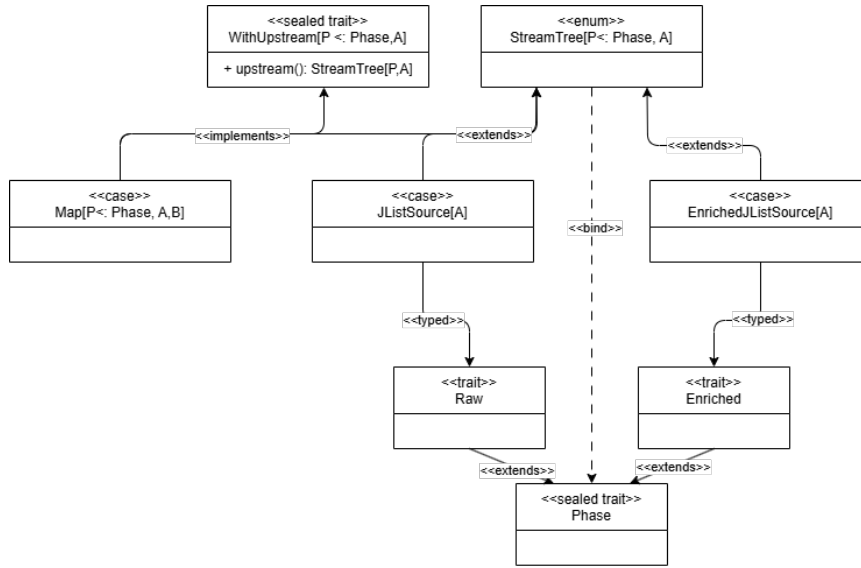


Figura 4.4: Struttura semplificata della Stream IR.

Gli operatori che non modificano la propria rappresentazione tra le due fasi sono parametrizzati direttamente tramite P , Ad esempio Map:

```
1 case Map[P <: Phase, A, B](upstream: StreamTree[P, A], ...) extends
  StreamTree[P, B]
```

Si noti come la fase del nodo impone anche la fase del relativo *upstream* garantendo la correttezza dell'intera struttura dati

Gli operatori la cui struttura viene invece effettivamente estesa hanno varianti specifiche per fase. Un esempio dedicato al `flatMap` è riportato nella sezione a seguire.

4.3.2 Rappresentazione di flatMap

Si consideri, ad esempio, la pipeline:

```
1 FusedStream
2   .from(values)
3   .flatMap(x =>
4     FusedStream
5       .from(makeValues(x))
6       .filter(y => y > 0)
7       .map(y => x + y)
```

```

8 )
9 .collect(Collector.toList)

```

Dopo il parsing, il nodo `FlatMap` contiene due distinti sottoalberi: `upstream`, che rappresenta lo stream esterno, e `innerTree`, che rappresenta la pipeline costruita all'interno della funzione di `flatMap`.

La struttura risultante può essere schematizzata come segue:

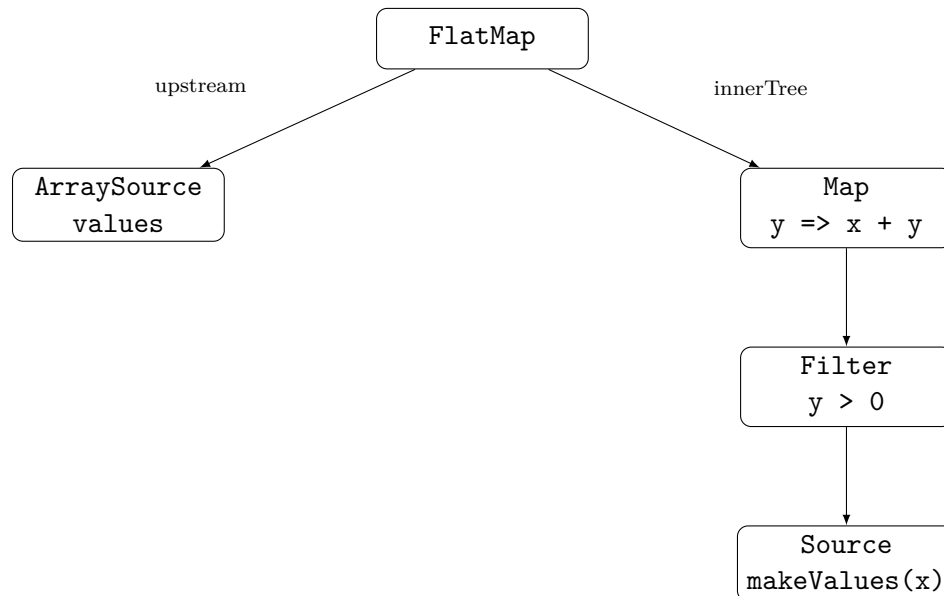


Figura 4.5: Rappresentazione di un'operazione `flatMap` nella Stream IR.

In questo caso, il riferimento all'elemento dello stream esterno costituisce un problema: l'espressione `x + y` appartenente alla pipeline interna dipende dal valore di `x`, che sarà disponibile solamente durante l'esecuzione dello stream esterno.

Per rappresentare questa dipendenza, durante il parsing viene creato un simbolo `elemSymbol` che svolge il ruolo di binder verso l'elemento esterno. La funzione passata a `flatMap` viene beta-ridotta sostituendo il suo argomento con tale simbolo; il parser può quindi analizzare ricorsivamente l'espressione risultante e ottenere `innerTree` come una normale Stream IR.

La trasformazione è:

$$x \Rightarrow \text{Stream}(x) \quad \longrightarrow \quad \text{Stream}(\text{elemSymbol}).$$

Per potere gestire lo stream interno al flat map il nodo `FlatMap` deve però poter "crescere" durante la fase di enrichment, incorporando anche le dichiarazioni e materializzazione di variabili del loop interno. Il nodo prodotto dal parser, appartenente alla fase `Raw` è definito come:

```

1 case FlatMap[A, B](
2   upstream: StreamTree[Raw, A],
3   innerTree: StreamTree[Raw, B],
4   elemSymbol: Symbol,
5   innerDeclarations: List[Statement]
6 ) extends StreamTree[Raw, B]

```

Durante la fase di enrichment il nodo viene trasformato in:

```

1 case EnrichedFlatMap[A, B](
2   upstream: StreamTree[Enriched, A],
3   innerTree: StreamTree[Enriched, B],

```

```

4     elemSymbol: Symbol,
5     innerDeclarations: List[Statement],
6     innerMaterialized: List[ValDef],
7     predicates: List[Expr[Boolean]]
8 ) extends StreamTree[Enriched, B]

```

Entrambi i sottoalberi appartengono ora alla fase **Enriched** e il nodo contiene due nuovi campi, **innerMaterialized** e **predicates**. Il primo raccoglie le definizioni che devono essere materializzate per ogni istanza della pipeline interna, mentre il secondo contiene i predicati che devono essere verificati durante la sua esecuzione.

4.4 Rappresentazione per la Code Generation

La Stream IR descritta nella sezione precedente mantiene una rappresentazione di alto livello della pipeline e dei suoi operatori. Tale struttura è particolarmente adatta alle analisi specifiche del dominio degli stream, ma non costituisce una rappresentazione conveniente per la generazione del codice.

Per questo si introduce una seconda rappresentazione intermedia, *Operation IR* (OP IR). La traduzione verso questa rappresentazione costituisce un processo di *lowering*: operatori di alto livello vengono eliminati e sostituiti da costrutti che descrivono esplicitamente il modello di esecuzione della pipeline. A differenza della Stream IR, la OP IR non modella quindi il dominio degli stream.

La rappresentazione è organizzata attorno a due categorie fondamentali: *Values*, che descrivono espressioni che producono un valore, e *Operations*, che descrivono azioni e strutture di controllo. Un programma nella OP IR è una sequenza di operazioni e un valore che ne costituisce il risultato:

```

1 case class Program[T](operations: List[Op], result: Value[T])

```

Questo ulteriore linguaggio intermedio permette di suddividere la generazione del codice in due fasi con responsabilità distinte. La prima opera sulla rappresentazione logica della pipeline e, sulla base degli operatori presenti, della sorgente, dell'operazione terminale e delle proprietà ricavate durante l'analisi, come la cardinalità e l'allineamento degli indici, determina la strategia di esecuzione più adatta e la esprime attraverso la OP IR. La seconda fase è invece meccanica: opera su un insieme ristretto di primitive della OP IR e ne effettua il lowering nei corrispondenti costrutti Scala. Le decisioni relative a come implementare la pipeline sono quindi prese prima della generazione effettiva del codice.

4.4.1 Valori

I *Values* rappresentano espressioni che producono un risultato e sono utilizzati come operandi dalle operazioni della OP IR. La gerarchia è composta da innanzitutto valori elementari, come costanti o riferimenti a Simboli e da valori composti che descrivono operazioni su altri valori come confronti, operazioni aritmetiche o applicazioni di funzioni, sono inoltre presenti valori specifici come accessi a array o iteratori.

Un **Value** descrive quindi un'espressione che può essere incorporata all'interno di una **Op**. Ad esempio, la condizione di un ciclo **While** è rappresentata come un **Value[Boolean]**, mentre il valore assegnato da una **AssignVal** è a sua volta rappresentato tramite un **Value**. Un esempio parziale della gerarchie è presente in figura 4.6

Ogni valore possiede l'informazione relativa al tipo che rappresentano, utilizzato durante la successiva generazione del codice.

```

1 sealed trait Value[T] {def valueType: Type[T]}

```

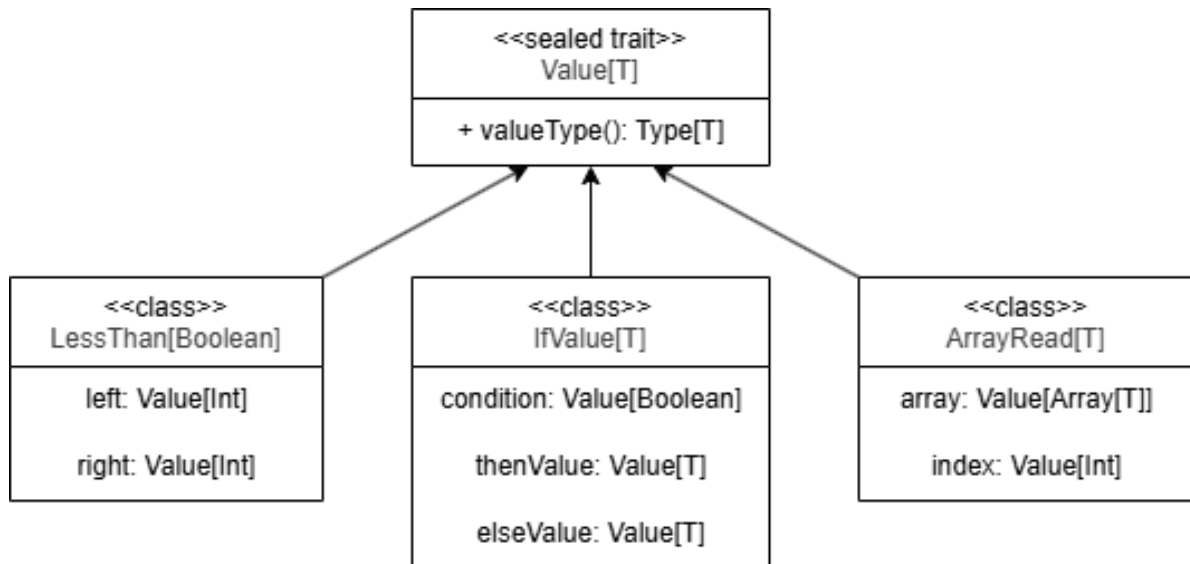


Figura 4.6: Struttura semplificata della Stream IR.

```

2
3 final case class LessThan(...) extends Value[Boolean] {
4   override val valueType: Type[Boolean] = Type.of[Boolean]
5 }
6
7 final case class IfValue[T](
8   ...
9 )(using val valueType: Type[T]) extends Value[T]
10
11 final case class ArrayType[T](
12   ...
13 )(using val valueType: Type[Array[T]]) extends Value[Array[T]]
  
```

Il parametro di tipo permette di esprimere staticamente i vincoli, a livello di tipi tra le diverse primitive della IR. Per esempio in `LessThan`, entrambi gli operandi devono essere valori interi, mentre il risultato è necessariamente un `Value[Boolean]`.

La sola informazione contenuta nel parametro generico non è però sufficiente per la successiva generazione del codice, ogni `Value[T]` espone quindi anche un `Type[T]`, attraverso il campo `valueType`.

Si realizza così una forma di *type capture*: oltre a conoscere staticamente che un nodo produce un valore di tipo `T`, la IR conserva una rappresentazione del tipo utilizzabile dalle Scala Quotes. Per tipi noti in maniera statica, per valori parametrici viene invece propagata attraverso un parametro contestuale `Type[T]`.

TODO: sposta in lowering e linka Questa informazione viene utilizzata durante il lowering per costruire espressioni Scala correttamente tipizzate. Il code generator può infatti recuperare il `Type` associato a un `Value` e renderlo disponibile come `given Type[T]` durante la costruzione delle `Expr[T]`. In questo modo la OP IR conserva le informazioni di tipo necessarie alla generazione senza dover ricostruire il tipo a partire dalla struttura del nodo.

4.4.2 Operazioni

Le *Operations* rappresentano invece le azioni che costituiscono il corpo del programma generato.

La OP IR comprende operazioni elementari per la dichiarazione e l'aggiornamento dello stato, come `Declare`, `AssignVal` o `Inc`. Il controllo del flusso è espresso esplicitamente attraverso `If` e `While`.

Sono inoltre presenti operazioni specializzate per le strategie di materializzazione utilizzate dalla libreria. `ArrayWrite`, ad esempio, rappresenta direttamente la scrittura di un elemento in un array, mentre `DynamicArrayBuilderAdd` rappresenta l'inserimento in un buffer la cui dimensione non è nota staticamente.

Capitolo 5

Sviluppo

5.1 Parsing

La fase di parsing ha l'obiettivo di trasformare l'albero sintattico tipizzato prodotto dal compilatore Scala in una `StreamIR` (Sezione 4.3) e di estrarre, a partire dal collector (Figura 4.3), la corrispondente `CollectionStrategy`. Il parsing della pipeline e quello dell'operazione terminale sono mantenuti separati e vengono successivamente combinati nella rappresentazione utilizzata dalle fasi successive.

5.1.1 Parsing della stream

Data una fused stream come:

```
1  val result = FusedStream
2    .from(numbers)
3    .filter(i => i % 2 == 0)
4    .skip(1)
5    .map(i => i * 10)
6    .collect(summing)
```

l'obiettivo della fase di parsing è ricondursi a una `StreamIr` del tipo:

$$\begin{aligned} \text{ArraySource[Int]} &\xrightarrow{\text{filter:}(Int \rightarrow \text{Boolean})} \text{Filter[Int]} \\ &\xrightarrow{\text{skip:}Int} \text{Skip[Int]} \\ &\xrightarrow{\text{map:}(Int \rightarrow Int)} \text{Map[Int, Int]} \end{aligned}$$

(nota, non è presente il collector in quanto trattato a parte, vedi 5.1.2)

Il parsing è effettuato analizzando l'AST tipizzato esposto dalla reflection API, in particolare il codice in Listing 5.1.1 produce:

Listing 5.1: Rappresentazione ad albero dell'AST di Scala

```
1  Inlined(
2    None,
3    Nil,
4    Apply(
5      TypeApply(
6        Select(
7          Apply(
8            Select(
9              Apply(
10               Select(
11                 Apply(
```

```

12         TypeApply(
13             Select(Ident("FusedStream"), "from"),
14             List(Inferred())
15         ),
16         List(Ident("numbers"))
17     ),
18     "filter"
19 ),
20 List(
21     Block(
22         List(
23             DefDef(
24                 "$anonfun",
25                 List(TermParamClause(List(ValDef("i", Inferred(),
26                     None))))),
27                 Inferred(),
28                 Some(
29                     Apply(
30                         Select(
31                             Apply(
32                                 Select(Ident("i"), "%"),
33                                 List(Literal(IntConstant(2)))
34                             ),
35                             "=="
36                         ),
37                         List(Literal(IntConstant(0)))
38                     )
39                 )
40             ),
41             Closure(Ident("$anonfun"), None)
42         )
43     ),
44     "skip"
45 ),
46 List(Literal(IntConstant(1)))
47 ),
48 "map"
49 ),
50 List(Inferred())
51 ),
52 List(
53     Block(
54         List(
55             DefDef(
56                 "$anonfun",
57                 List(TermParamClause(List(ValDef("i", Inferred(), None)))),
58                 Inferred(),
59                 Some(
60                     Apply(
61                         Select(Ident("i"), "*"),
62                         List(Literal(IntConstant(10)))
63                     )
64                 )
65             )
66         ),
67         Closure(Ident("$anonfun"), None)
68     )
69 ),
70 )
71 )
72 )

```

Rappresentazione dei nodi: ParsedTree

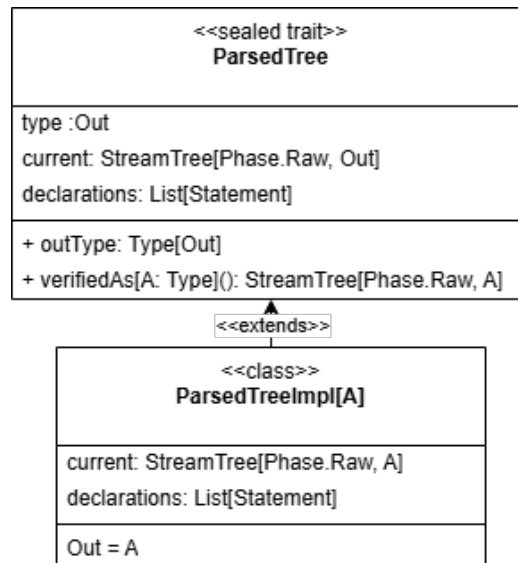


Figura 5.1: Struttura per parsed tree

Il trait `ParsedTree` rappresenta il risultato intermedio del parsing senza richiedere che il tipo di output sia noto al chiamante. Questo è necessario perché il tipo prodotto da una pipeline non è noto a priori, ma viene "scoperto" durante il parsing.

Per rappresentare questo tipo "nascosto", `ParsedTree` utilizza un abstract type member `Out`. Il tipo concreto viene fissato al momento della costruzione dell'implementazione, quando il tipo è noto. In questo modo il parser può esporre semplicemente un `ParsedTree`, mantenendo però internamente la correlazione tra il tipo dell'albero `current: StreamTree[Phase.Raw, Out]` e il relativo tipo di output.

Il tipo nascosto può essere utilizzato nelle successive fasi di parsing tramite un path-dependent type, ad esempio `type In = upstream.Out`. Il tipo precedentemente nascosto diventa così nuovamente utilizzabile per tipizzare operazioni successive, come una funzione `In => Boolean` per `filter` oppure `In => Out` per `map`.

Infine, `outType: Type[Out]` conserva l'evidenza reificata del tipo nascosto, necessaria per interagire con le API di `scala.quoted`, ad esempio per convertire un `Term` in una `Expr[In => Boolean]`.

Il riconoscimento delle espressioni

Il riconoscimento delle espressioni è effettuato tramite pattern matching

Listing 5.2: Pattern Matching sul Syntax Tree in Scala

```

1 private def parseTerm(expr: ir.quotes.reflect.Term): ParsedTree =
2   expr match
3     // In caso di espressioni inlined analizza l'espressione sottostante
4     case Inlined(_, _, inner) => parseTerm(inner)
5     case Apply(TypeApply(NamedMethod("from"), _), List(arr)) => ...
6     case Apply(TypeApply(NamedMethod("of"), _), List(arr)) => ...
7     case Apply(Select(upstream, "filter"), List(predicate)) => ...
8     case Apply(Select(upstream, "skip"), List(n)) => ...
9     case Apply(Select(upstream, "limit"), List(n)) => ...
10    case Apply(TypeApply(Select(upstream, "map"), List(outType)),
11      List mapper)) => ...
11    case Apply(TypeApply(Select(upstream, "flatMap"), List(outType)),
12      List mapper)) => ...
  
```

```

12 case Apply(Select(upstream, "parallel"), _) => parseTerm(upstream)
13 // In caso di un blocco accumula le dichiarazioni e continua con il parsing
14 case Block(blockStatements, expr) => parseTerm(expr) match {
15   case ParsedTreeImpl(tree, decls) => ParsedTreeImpl(tree, blockStatements
16     ++ decls)
17 }

```

Esempi di parsing specifico: filter

Ogni caso poi tratta le informazioni ricevute ad esempio un filter è implementato come:

Listing 5.3: Parsing di un'operazione filter

```

1 case Apply(...) => appendFilter(parseTerm(upstream, predicate))
2
3 private def appendFilter(upstream: ParsedTree, predicateTerm: Term):
4   ParsedTree = {
5     type InOut = upstream.Out
6     given Type[InOut] = upstream.outType
7     val predicate = predicateTerm.asExprOf[InOut => Boolean]
8     val filter = Filter[Phase.Raw, InOut](upstream.current, predicate,
9       upstream.outType)
10    ParsedTreeImpl[InOut](filter, upstream.declarations)
11  }

```

Qui emerge un pattern ricorrente nel progetto: un tipo dipendente viene associato alla relativa evidenza reificata richiesta dalle API di `scala.quoted`. Il tipo di output del nodo precedente viene definito localmente tramite `type InOut = upstream.Out`. Viene quindi resa disponibile un'istanza `Type[InOut]` a partire da `upstream.outType`. In questo modo il predicato e l'albero `upstream.current` condividono staticamente lo stesso tipo `InOut`, senza ricorrere a cast.

Esempi di parsing specifico: flatMap

Un caso più complesso è dato da `flatMap`. A differenza `filter` la funzione passata a `flatMap` produce a sua volta una `Stream`. Tale stream deve quindi essere estratta dalla funzione e sottoposta ricorsivamente alla stessa procedura di parsing.

Si consideri, ad esempio:

```

1 FusedStream
2   .from(numbers)
3   .flatMap(x =>
4     FusedStream.from(elements(x)).filter(y => y > 0)
5   )

```

Il parametro `x` della funzione rappresenta l'elemento prodotto, a runtime, dall'`upstream`. Durante l'espansione della macro tale valore non è ancora disponibile; è tuttavia necessario per analizzare staticamente la stream interna. A questo scopo viene creato una variabile di binding di tipo `In`, ovvero il tipo di output dell'`upstream`:

```

1 type In = upstream.Out
2 given Type[In] = upstream.outType
3 val flatMapBinder = createConstantIn
4 val elemRef = Ref(flatMapBinder).asExprOf[In]

```

Il simbolo non contiene quindi il valore effettivo dell'elemento, ma costituisce un placeholder tipizzato che lo rappresenta all'interno della IR. La funzione ricevuta dal `flatMap` viene dapprima convertita nella corrispondente espressione tipizzata:

```

1  getType(outputType) match {
2    case '[out] =>
3      val function = functionTerm.asExprOf[In => Stream[out]]
4      ...
5  }

```

Quello che si ottiene è una funzione del tipo:

$\text{In} \rightarrow \text{Stream}[\text{Out}]$.

A questo punto la funzione viene beta-ridotta contro il riferimento al binder:

```

1  val innerStreamExpr: Expr[Stream[out]] =
2  Expr.betaReduce {'{ $function($elemRef) }}

```

Ciò che si ottiene è quindi la stream interna che viene quindi sottoposta ricorsivamente a `parseTerm`. Si ottengono quindi due sottoalberi: l'upstream esterno e la stream interna prodotta dal `flatMap`. Il nodo della IR mantiene inoltre il simbolo del binder che collega i due: Durante la successiva fase di code generation il binder verrà associato al valore dell'elemento corrente prodotto dall'upstream. In questo modo la stream interna può essere analizzata e ottimizzata durante la compilazione pur dipendendo da un valore che sarà disponibile soltanto a runtime.

5.1.2 Collector

La collection strategy

La fase terminale di una stream è rappresentata internamente attraverso una `CollectionStrategy`. Questa astrazione descrive la strategia con cui gli elementi prodotti dalla pipeline devono essere consumati e aggregati per ottenere il risultato finale.

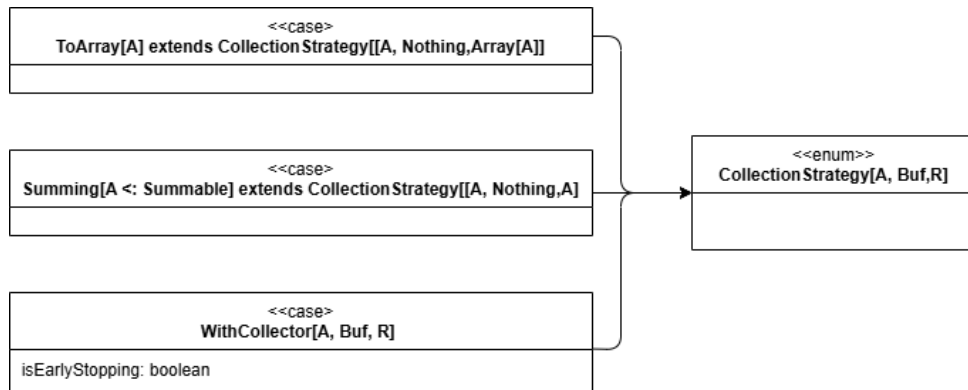


Figura 5.2: Diagramma UML delle classi per la gerarchia `CollectionStrategy`.

L'introduzione di questa rappresentazione permette di separare il collector espresso dall'utente nella DSL dalla modalità con cui il compilatore genera il codice relativo all'operazione terminale. In particolare, i collector possono essere suddivisi in due categorie.

La prima comprende i *collector concreti*, che forniscono direttamente l'implementazione delle operazioni necessarie alla raccolta. Tali operazioni vengono quindi invocate dal codice generato durante l'esecuzione della stream.

La seconda comprende invece i *collector opachi*. Questi non costituiscono l'implementazione effettivamente utilizzata a runtime, ma agiscono come placeholder tipati all'interno della DSL. Il compilatore li riconosce durante il parsing e li sostituisce con una `CollectionStrategy` specializzata, la cui implementazione è definita direttamente all'interno della libreria. Appartengono a questa categoria, ad esempio, i collector utilizzati per le operazioni `toArray` e `summing`.

Questa distinzione consente quindi di specializzare le operazioni terminali note alla libreria, evitando di rappresentarle necessariamente attraverso il protocollo generale di un collector.

Parsing del collector

Il primo passo consiste nell'analizzarne il tipo, la `TypeRepr` associata all'espressione viene confrontata con i simboli dei collector opachi della libreria. Se è identificato uno dei collector opachi, il parser costruisce direttamente la strategia specializzata corrispondente, in questo modo, `ToArray` e `Summing` entrano nell'IR non come collector generici, ma come operazioni terminali note al compilatore. Nel caso generale, invece, il collector fornito dall'utente deve essere preservato, viene quindi estratta la *stop policy* per determinare staticamente se il collector supporti lo short-circuiting. Il risultato di questa analisi viene infine incorporato nella strategia generica `WithCollector`. La trasformazione può essere rappresentata come:

$$\text{ToArrayCollector}[A] \longrightarrow \text{ToArray}[A]$$

$$\text{SummingCollector}[A] \longrightarrow \text{Summing}[A]$$

$$\text{CollectorBase}[A, \text{Buf}, R] \longrightarrow \text{WithCollector}[A, \text{Buf}, R]$$

Nota la tipizzazione su `TerminationPolicy` a questo punto viene persa, è infatti utilizzata solo per definire la grammatica della dsl e viene convertita nel valore booleano `isEarlyStopping` dentro `WithCollector`

5.1.3 Parsing della configurazione statica

La libreria effettua un'ulteriore operazione di parsing relativa alla configurazione di compilazione. A differenza del parsing della stream, basato sull'analisi del TASTy, in questo caso l'estrazione viene effettuata mediante quoted pattern matching. Poiché la configurazione deve essere interamente nota a compile time, i valori dei suoi campi devono poter essere estratti staticamente dall'espressione. Sono quindi accettate istanziazioni di `CompileConfig` i cui argomenti siano costanti note in fase di compilazione, come mostrato dal seguente `FromExpr`:

Listing 5.4: Unapplier con quoted pattern matching per configurazione statica

```

1  given FromExpr[CompileConfig] with {
2    def unapply(expr: Expr[CompileConfig])(using Quotes):
3      Option[CompileConfig] = {
4        expr match {
5          // CompileConfig(unsafe, logging)
6          case '{ CompileConfig($u, $l) } =>
7            for {
8              unsafe <- u.value
9              logging <- l.value
10             } yield CompileConfig(unsafe, logging)
11
12          // new CompileConfig(unsafe, logging)
13          case '{ new CompileConfig($u, $l) } =>
14            for {
15              unsafe <- u.value
16              logging <- l.value
17             } yield CompileConfig(unsafe, logging)
18          }
19        }

```

5.2 Enrichment

5.3 Traduzione in OPIr

5.4 Lowering

5.5 Testing

5.6 Benchmark

Capitolo 6

Retrospettiva

Bibliografia

- [1] Shayan Najd and Simon Peyton Jones. Trees that grow. *Journal of Universal Computer Science*, 23(1), 2017.